

Chapter One: Our “Product Vision”

CO-SIGNERS: Evan Myran

Cadence Stewart

Rachel Roach

Andrew Martens

Tyler Safranski

Product Vision

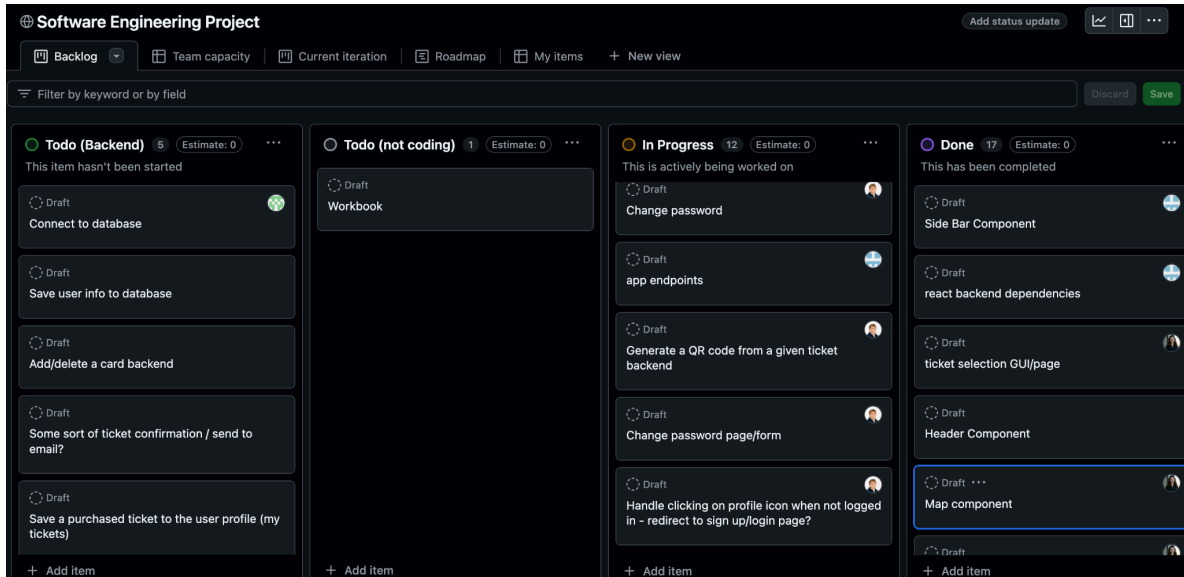
FOR commuters, travelers, and small businesses WHO are experiencing difficulty purchasing and managing their train tickets, THE Train Track provides a web-based application THAT gives users the ability to purchase, manage, and scan train tickets with minimal complexity. UNLIKE other ticketing applications, OUR application offers an all-in-one application allowing users to purchase, store, and scan their train tickets straightforwardly and simplistically.

Agile Workflow

Our team had a hybrid approach to Agile workflow and SCRUM. In most cases, our team collaborated online due to scheduling conflicts during the week. This method allowed us to have daily SCRUM sessions even when schedules collided. When necessary, our team organized weekend meetings. This occurred whenever we needed all members' input or help on larger aspects of the project like the database integration or backend workflow.

Team Organization & Scrum Implementation

For team organization, we used a GitHub backlog to assign tasks and we used a Discord server to communicate effectively with each other. When we started the project we worked through as many steps of our project as we could think of and added it to the backlog. Then we split the tasks up amongst our group members and began working on our parts of the project. We kept our group members updated on our progress by categorizing each task as ‘To do’, ‘In Progress’, or ‘Done’. We tracked our overall progress by seeing how many items on the backlog were completed or in progress.



We implemented Scrum in our way, which fits the model of this class and the trajectory of the project. Our development team sprints ended up being the periods in between in-class presentations. We did this because these were easily identifiable days to have features completed that worked with our class schedule. Our scrum daily meeting was usually online communication through Discord as we did not have time for our group members to meet daily. We used this time on Discord to catch up on any progress we had made and any updates to the main GitHub repository. Our sprint planning ended up occurring in a class period or in an online meeting that we hosted on the weekend. This allowed our team to converse on the features that needed development and what each group member needed to finish to advance the development of the project. Sprint reviews occurred online for our group on our discord where we would discuss what changes needed to be made to each section of the project after presenting or meeting online.

‘Sprint’ Description

As described above, our sprint format was structured around the class setup and class check-ins. We used the time between in-class presentations as our “sprints” as these were periods with clearly defined end times for feature development. These were times when our team knew we needed to have updates finished and polished off and knew where we needed to work for future sprints. We would meet at any opportunity we could in class for the full amount of time. We would also meet on the weekends at noon on Sunday sometimes online or in person in the computer science room. Depending on the length of time between in-class presentations would determine the number of times our group met in individual sprints. No matter how many times we met, we still maintained constant communication in our Discord server where we updated our team on new updates and project progress.

Development Process & First Experience with Agile Development

The development process was a little rough when we first started. Everyone had different levels of comfortability using the source control technologies, and it took us a while to set up our GitHub repository and GitHub project (backlog). Once we all got GitHub and Discord set up, the development process got much smoother. We all used our branches which allowed each member to work on separate features without stepping on each other's toes, and if anyone ran into an issue they could send a message through Discord to address the problem without needing to wait until the next SCRUM. As mentioned above, we took a hybrid approach to our agile development because of each individual's busy schedule. Due to this approach, our SCRUM meetings were often held informally through Discord allowing us to communicate effectively with each other while keeping our SCRUM flexible. While our first experience with agile development wasn't traditional, it catered to the busy lives we lived as college students. This schedule worked the best for our group making our first experience in an agile workflow a beneficial experience.

Chapter Two: Narrative on Software & Hardware Stack

CO-SIGNERS: Evan Myran

Cadence Stewart

Andrew Martens

Tyler Safranski

Rachel Roach

IDE Program

For our development environment, most of us chose to develop in Visual Studio Code (VSCode). VSCode has features that were helpful for this project, such as integration with Git and Github and an integrated terminal. Familiarity was the primary reason for choosing the IDE since most of us had used VSCode. There was one member on the team who had not worked in VSCode previously and chose to work directly on the GitHub online interface. This was useful because it's easier to pull changes into the terminal and see visually new updates on localhost.

Programming Languages

The main programming language we used was Javascript. Javascript was used since it integrates with both our frontend development (React) and our backend development (Node). CSS and HTML were used for formatting/styling as well.

Code Base Management - <https://github.com/cmstewa19/Software-Engineering-Project>

For code base management, we used Git for version control and Github as our remote repository. Each of us cloned the repository to our local machines which allowed us to all work on the same code base. For each new feature, we made a new branch, and when this branch was completed it was merged into the main branch via a pull request. We did not require approval from other team members for the pull requests because of our asynchronous developing schedules but did notify each other via Discord when a new feature had been completed.

Development Target Description

Our development target was originally an Android application, but developing for Android would have required writing in Kotlin which none of us had prior experience with. Due to that conflict, we changed our development target to a browser-based web application. We believe this change still allows us to develop a product that stays in line with our product vision. We stayed in line with our project vision by thinking about how a user might interact with the application if they were accessing it on a web browser on their phone, emulating a scenario where users may be at a train station and don't have a computer readily available to get access to our web service. This is reflected through our personas, scenarios, and stories where potential users are created to show what our audience seeks in our platform and how they might interact with it.

Chapter Three: Personas, Scenarios, and Stories

PRIMARY AUTHOR: **Rachel Roach**

CO-SIGNERS: Evan Myran
Cadence Stewart
Andrew Martens
Tyler Safranski

Persona 1:

Michael is a 32-year-old man living in Washington, DC. He double majored in Business Administration and Finance at Georgetown University before working as a contracted aid for small businesses around the country, helping them get their business running and managing their finances. Because of the nature of his job, he is an experienced traveler both in and outside his professional life making him well-versed in different methods of travel. Michael has a hard time flying due to airline websites being confusing to operate and airlines being expensive. For this reason, he's interested in exploring other means of travel and wants to investigate sites like "Train Track" to see how well they integrate into his lifestyle. Michael has an average level of technology knowledge but is extremely familiar with how booking, airline, and other travel sites function due to the nature of his job.

Persona 2:

Janet is a 42-year-old woman who currently lives in New Orleans, Louisiana. She has a minimal understanding of technology outside of the airfare sphere because she prefers to live in the moment as opposed to having a large presence online. After she graduated high school, she immediately pursued her goals as a flight attendant and traveled around the world, doing so through the airline. She enjoys face-to-face interaction wherever she goes because she believes it's essential for building connections with people. She is not interested in other forms of travel because she feels it threatens her job, and she also thinks new technology surrounding booking travel is unnecessary because the booking methods that currently exist do the job just fine. For these reasons, she finds "Train Track" ineffective in doing its job, and would only be interested in utilizing its mechanics when other forms of travel aren't available; however, her familiarity with airfare makes her unlikely to try different forms of travel even when there are no flight options.

Persona 3:

Robert is a 67-year-old man originally from Kansas City, Kansas who recently moved to Tampa, Florida to live out the remaining years of retirement with his wife Carol. He gained his undergraduate degree at the age of 31 majoring in Communications at Concordia University because he felt that he would receive more opportunities to grow in his career if he had a degree. Robert is a very inexperienced traveler. He lived the majority of his life in Kansas City and only

moved to Tampa because his wife encouraged him to move. He has a surface-level understanding of technology due to his former line of work—Marketing—requiring him to be up-to-date with developing technology and to observe trends online that could be beneficial to their marketing strategies. Because of his combined lack of interest in travel and average knowledge level of technology, Robert would not be inclined to use “Train Track.” The only instance where he’d use the website is if he and his wife go on vacation since he prefers slower forms of travel where he can relax instead of faster-paced travel options like flight. If he were to use the site, he would want features that are simple to understand so he doesn’t have to deal with overwhelming complicated technology.

Persona 4:

Samantha is a 21-year-old woman currently living in Seattle, Washington. She recently graduated with a computer science degree at the University of Washington. Because of her level of expertise in the computer science field, she has a deep level of understanding surrounding technology, and she often travels from state to state to speak on panels at software conventions while interviewing clients for computer science-based fields around the country. Currently, she travels via plane but is interested in exploring other means of transportation that are simpler to understand and use compared to planes or buses. Additionally, she is working on a project that analyzes travel data of trains specifically and is looking for a platform to incorporate into her research. Because of this, she is very interested in “Train Track” because the train data can easily be incorporated into her work and it provides an ease-of-travel experience that she has desperately been looking for.

Scenario:

Michael is a 32-year-old freelancing business aid who travels around the country helping small businesses get off the ground. He lives in Washington DC and double majored in business and finance at Georgetown University. He’s seeking new methods of transportation because even though traveling by plane is quickest, he finds the process of booking a ticket, going through the airport, and finding his terminal extremely difficult and confusing to operate around. He needs not only a new method of transportation but a new method of buying the tickets for said transportation.

In doing so, he discovers the Train Track website that offers many different actions for its users that make their lives easier. As he’s exploring the site and planning to buy a train ticket, he searches for trains running from Washington Station to Moynihan Train Hall with the search feature available from the home page. From there, he will need to visually see the path the train is traveling and the number of available seats on the train. This can be done by clicking the ticket he previously searched for where both visual aids can be accessed, making it simple to understand where he’s going and sitting on the train. Additionally, because he travels a lot, he could utilize a method where he can keep track of the past tickets he’s bought so he can look at

where he's traveled throughout the year for a personal end-of-year summary. For now, he keeps track of his current purchases in the Notes app on his phone.

When it comes to safety, Michael accepts no shortcuts. He needs a program that is tightly secured and one that can safely hold his financial information to prevent theft from occurring from the account. Through Train Track, he can easily secure his account by creating a profile connected to his email and by providing a secure password that can be viewed at any point in time through the profile page on Train Track. A feature he could find useful is a function that can safely and securely change his password to his account in case something were to happen.

Story: As a traveler, I need a way to easily book train tickets and manage the tickets I've already purchased.

Justification for the Number of Personas, Scenarios, and Stories:

We decided on a total of four personas to depict our product. The first one is representative of the average person who would be interacting with and interested in our website. Ideally, they would be well-versed in travel and have an average understanding of technology allowing them to be familiar with operating ticket booking sites. They would want to investigate new forms of travel and the websites related to that, and they would explore different options like Train Track that could make their life easier.

The second persona represents someone who frequently travels, but is apprehensive and uninterested in exploring new travel and booking websites. These people would need/want to travel frequently but prefer the familiarity and quickness that other forms of travel offer compared to trains and the services Train Track offers. If other methods of travel weren't available, these people might be interested in exploring Train Track, and with features similar to that of air travel like seat booking and a direct path of travel visible to the user, they could easily understand the functionality surrounding it. Outside of that specific instance, however, they would be apprehensive about Train Track.

Persona three on the other hand represents someone who does have an average level of knowledge in technology but isn't interested in travel itself and is thus not inclined to utilize websites like Train Track. They wouldn't be interested in using the site because they don't have a specific need for it. People like Robert would only interact with the site when they need to make it important to emphasize how easy the platform is to use for new users who don't have prior experiences with sites like Train Track and who don't travel frequently.

The final persona represents someone who's enthusiastic about Train Track and is very interested in learning more about the platform. They would have an advanced understanding of travel, ticket sites, and technology that incentivizes exploration of new forms of travel and the sites that make that type of travel easier. This persona was chosen to show a user who willingly experiments with the program and shows someone willing to go out of their way to learn more about what Train Track has to offer.

One scenario was chosen to represent Train Track because it delves into more detail surrounding what a potential user would want out of our system. It was based on the first persona because it is the most accurate in representing the average user of Train Track. It emphasizes the features that would be most useful to people like him, and it makes him more personal, reminding our team that we are creating this product for real people instead of solely customers. It centers the product around a central goal and keeps the team in line with what users want out of the program.

A singular story was chosen to represent the overall goal of the program. This is because the singular story encapsulates a user's wants and needs with Train Track in a simple, short answer. In our case, the reason why a user would want to use a system like Train Track is because they seek a way to easily manage and purchase train tickets. Capturing the goal of average users makes it easier for our team to stay focused on a centralized goal: tickets and ticket management.

Chapter Four: Software Architecture

PRIMARY AUTHOR: **Evan Myran**

CO-SIGNERS: Rachel Roach
Andrew Martens
Tyler Safranski
Cadence Stewart

Chosen Architecture:

We followed a layered architecture approach to our project. This model helped us to encapsulate responsibilities, reduce dependencies, and enhance security. The layering of our web app components gave us inter-layer communication within the project, a clear separation of dependencies, and made the code flow very easy to follow. Each layer of our project is modular. In each layer, we have specific tasks such as user interface management (src/), backend services (server/), and shared resources (node_modules/). In our src folder, we had our pages and components folders. In each of these directories, we had a clear separation of every page of our web, and any functional parts on the page were separated. This allowed us to minimize errors as there was less code. This ensured components and pages could be independently updated or replaced.

We chose to use React to support our backend dependencies. We knew we were going to use a 3rd party system to build our backend as none of our group had had any experience with handling such a task on our own. We implemented Node.js to compile and run our local host. This program was the one that our group had the most experience with coming into this project and it is very simple to build and run. Node.js also can build itself into the GitHub structure which was the platform our group was using. This saved us a ton of time because we didn't have to worry about the file structure of our backend because it was self-installed. We tackled security measures like authentication, which are implemented as cross-cutting concerns, by implementing error handling code in both the front end (login and sign-up forms) and back end (API calls). Our error handling code consisted of tracking our console log (console.log()), data validation on every user input, and try statements on almost all blocks of code dealing with calling data. This was important because it gave our program greater security in that all flows of data were protected and checked in our program.

Visual depiction of architecture:

Software-Engineering-Project/

```
├─ node_modules/      # External dependencies managed by npm and react
├─ public/            # Static assets such as images and stylesheets for web pages
├─ server/            # Backend logic and server-side API handling for calling tickets and users
├─ src/               # Frontend source code, including React components and pages
└─ package-lock.json  # Version-lock file for npm dependencies.
```

```

├─ .gitignore      # File exclusions for version control, .env file with API keys
├─ package.json    # Project metadata and dependencies.
├─ README.md       # Documentation for project setup and usage.

```

Readable description of architecture:

Our fronted layer housed all of our UI components in our `src/` folder. This folder contained our React components, pages, client-side routing logic, a folder of all non-coded assets, and our `App.js` file which was responsible for all defined pages and paths in our program as well as formatting the application. Our public assets were stored in `public/` and delivered our static files. The idea of decoupling the UI logic into components improved our maintainability and supported scalability in our agile environment. It was very difficult for us to add features and pages as we worked. This was because allowing individual components to be tested and modified without affecting the entire system. Our backend layer, our servers, and APIs are located in our `server/` folder. It contains server-side code to manage requests, authentication, and data flow between the client and the database. Our `express.js` file is used for handling API requests, aligning with the project's emphasis on responsiveness and maintainability. Separating our server logic ensured the backend remained robust. It also gave us opportunities to enhance the performance and scale API endpoints as needed. All dependencies and version control for our project were structured in `package.json` and `package-lock.json` to ensure consistent development environments and dependency management. The rationale for centralizing configurations was to minimize deployment errors and support collaboration amongst our group.

System Qualities:

1. Responsiveness

The layered architecture ensured that each component operated independently. This led to an optimized response time because our code and data flow were very sectioned off. The frontend, built with React, ensured efficient rendering on our pages, while the backend, implemented using Node.js, minimized latency in handling API requests. Static assets in the `public/` folder further speed up the user experience as the data and files were in the directory. We only needed to call the AMTrak data and nothing else. Our group experienced quick load times and seamless interactions, meeting expectations for real-time performance if this were to transfer to a real server.

2. Reliability

Reliability in our project was achieved by creating modules in the system and separating them into separate layers. This reduced interdependencies and simplified our testing methods since we had our `console.log` reports and try statements/error handling. Clear interfaces and differences in paths between components ensure predictable behavior and easy error tracing in

our development. The outcome of all of this was that our group could confidently develop or modify individual components and pages without introducing system-wide errors.

3. Availability

Our backend services are designed to remain operational even under high traffic loads. Both React and Node.js are industry programs meant to support complex applications. So our scalability is supported by the decoupled architecture of these backend programs, allowing horizontal scaling of server instances if the user base grows. In this scenario of our project, this won't be the case as we won't leave the local host but in theory, this would be true. The outcome of this is the system maintains high uptime, ensuring that services are accessible whenever users or our group members need them.

4. Security

For security, we opted for a centralized security model that protects user data by enforcing authentication and authorization in the backend (server/). Additional security layers, such as input validation, try/catch statements, and error handling, are implemented across the program. Specifically in our programs that deal with user input and calling data. These are the weak spots in our program so we attempted to beef them up as much as possible to reduce the chances of security breakdowns. The outcome of our security choices was that we got a system that safeguards user information and prevents unauthorized access.

5. Usability

The React-based front end focuses on an intuitive user interface, separating presentation and logic for easy updates. This improves the usability with users as the logic and component-based sections of the program do not interfere with this presentation and styling these components and logic is very simple. We kept all of our pages simplistic and well-defined. All navigation buttons and the flow of the website are well-defined and easy to comprehend. Responsive designs and clear navigation enhance the user experience. We did this by simplifying the data flow and keeping code flows to a couple of programs in length. This also prevents errors and improves usability because response time is high as well as functionality. The outcome of our project is that our group and any users can efficiently access system features with minimal learning curve or errors.

6. Maintainability

Our group's use of a modular directory structure (e.g., src/, server/) ensured that components were isolated and self-contained. This allowed our group to easily identify and update specific parts of the program without disrupting other components. The outcome of this developmental feature was that future features and bug fixes could be implemented at an effort.

7. Resilience

For this final architecture feature, resilience, we incorporated mechanisms into our program to handle failures gracefully. Examples of this were input validation to prevent crashes and modular components that isolate potential issues. For example, if a backend API fails, the frontend can display meaningful error messages without freezing. This helped our group development tremendously because it was very easy to identify what sections of the code were having issues and allowed us to fix them quickly. The outcome of incorporating resilience into our program was our system continued to operate reliably, even in the face of partial failures/

Diagram:

node_modules/ dir:

..		
.bin	added forgot password page and attached it to the backend.	yesterday
js-tokens	Initial commit with Node and React setup	2 months ago
loose-envify	fix react-refresh error	3 days ago
react-dom	Initial commit with Node and React setup	2 months ago
react	Initial commit with Node and React setup	2 months ago
scheduler	Initial commit with Node and React setup	2 months ago
.package-lock.json	added forgot password page and attached it to the backend.	yesterday

The node_modules directory was where our backend structure was located. In this folder, we had our React back end, the Node.js, back end, and all of our JavaScript compilers. Keeping this directory sectioned off from the rest of the code allowed us to worry about developing our front end and letting the third-party systems we had built on take care of our back-end processing.

public/ dir:

..		
index.html	changed the default icon to our icon	last week
rail.ico	changed the default icon to our icon	last week

Our public/ directory housed our website icon and our indexing file for the website. These files were static files that were not assets but needed to be delivered into components on our website.

server/ dir:

..		
database.js	Created database.js for handling database creation	5 days ago
server.js	Update server.js add get-tickets endpoint	yesterday
tickets.db	fixed errors in purchaseTickets.js to make it compileable	yesterday

Our backend layer, our servers, database, and APIs, are located in our server/ folder. It contains server-side code to manage requests, authentication, and data flow between the client and the database. Separating our server logic ensured the backend remained robust. It also gave us opportunities to enhance the performance and scale API endpoints as needed

src/ dir:

..		
assets	Made QR code clickable and created UI for scanned ticket page	last week
components	Update checkoutButton.js reset	yesterday
pages	Update purchaseTickets.js find a way to get the user id	20 hours ago
services	Restructured BrowseTrains page & table	2 weeks ago
style	Added welcome text to home page to clearly see user islogg...	yesterday
utils	Restructured BrowseTrains page & table	2 weeks ago
App.js	added forgot password page and attached it to the backend.	yesterday
index.js	Basic React web page functioning.	last month

Our fronted layer housed all of our UI components in our src/ folder. This folder contained our React components, pages, client-side routing logic, a folder of all non-coded assets, and our App.js file which was responsible for all defined pages and paths in our program as well as formatting the application.

components/ dir and pages/ dir:

QRCode.js	ChangePasswordPage.js
SearchBar.js	browseTrainsPage.js
bookingSeatSelection.js	changePasswordPage.js
changePasswordForm.js	forgotPasswordPage.js
checkoutButton.js	homePage.js
checkoutForm.js	loginPage.js
divider.js	myTickets.js
forgotPasswordForm.js	profilePage.js
header.js	purchaseTickets.js
headerNoSidebar.js	scanTicketPage.js
loginButton.js	signupPage.js
navigationButton.js	successPurchase.js
profileHeader.js	trainInfoPage.js
seatMap.js	userTicketsPage.js
sidebar.js	
trainDetails.js	
trainRouteMap.js	
trainSearch.js	
trainTable.js	
usernamePasswordForm.js	

In our src folder, we had our pages and components folders. In each of these directories, we had a clear separation of every page of our web, and any functional parts on the page were separated. This allowed us to minimize errors as there was less code. This ensured components and pages could be independently updated or replaced.

Database, Environment, Tools, Languages:

In designing our software product, several key technology decisions were made, each driven by specific project requirements and team expertise. Below, I outline the decisions made, the technologies chosen, and the reasoning behind these choices.

1. Database

We selected Microsoft SQLite as our database. Our reason for choosing this db was because several of our group had experience with this database before. SQLite is also a serverless database which fits perfectly into the structure of our project because we were running on a local host and needed a product that did not need to be hosted. SQLite also is very compatible with Node.js, our backend construction, and even built documentation for the integration of these two that helped make it even easier for us to use.

2. Environment

Most of the team used Visual Studio Code (VSCode) as the development environment, while one member worked directly on GitHub. All of our projects were hosted on GitHub and all working versions were pushed and pulled from our main working versions. Several forks were created for development and testing. VSCode was chosen for its widespread familiarity among team members, offering a common platform with robust features like debugging, version control integration, and extensions tailored for JavaScript and React. The ability for one team member to work directly on GitHub accommodated varying levels of familiarity with development environments.

3. Tools/Languages

We used Node.js for our backend dependencies. Node.js is ideal for handling multiple simultaneous requests, ensuring that our backend can scale as the user base grows. This was important for our scalability. Another one of our chosen tools was JavaScript. We used JS as our main language across all aspects of our project. Using JavaScript across the backend and front end allowed for a consistent development experience, reducing the learning curve and simplifying integration. Here is a quick summary of all of our major front-end dependencies which were React, QRCode, React Google Maps, Axios, and Stripe. React was the core framework for our front end, with additional libraries such as QRCode for generating QR codes, React Google Maps for map integration, Axios for API calls, and Stripe for payment processing.

Chapter Five: Reliable Programming

PRIMARY AUTHOR: **Cadence Stewart**

CO-SIGNERS: Evan Myran
Andrew Martens
Tyler Safranski
Rachel Roach

Reliability, User Experience, and Maintainability Considerations

We prioritized several key attributes when making our design decisions, as discussed in Chapter 4 of this workbook. These include reliability, availability, security, resilience, responsiveness, usability, and maintainability. These attributes were based on the design considerations presented by the textbook author under Reliable Programming. We implemented various strategies to prevent errors and handle them gracefully when they do occur, so as to not disrupt the user experience or impact the reliability of the application.

The screenshot shows a code editor with two panels. The left panel displays the JavaScript file `trainInfoPage.js`, and the right panel displays the CSS file `trainInfoPage.module.css`.

JavaScript Code (trainInfoPage.js):

```

71
72 return (
73   <div className={styles.body}>
74     <Header />
75
76     <div className={styles.leftSection}>
77       <div className={styles.trainDetails}>
78         <TrainDetails trainData={trainData} />
79
80       <div className={styles.trainRouteMap}>
81         <TrainRouteMap
82           origin={trainData.origin}
83           destination={trainData.destination}
84           center={mapCenter}
85         />
86       </div>
87     </div>
88
89     <div className={styles.seatMapAndBooking}>
90       <div className={styles.trainSeatMap}>
91         <SeatMap
92           trainCode={trainData.trainCode}
93           seatRows={12}
94           seatCols={5}
95           bookedSeats={trainData.bookedSeats}
96           onSeatsSelected={handleSeatSelection}
97         />
98       </div>
99
100       <div className={styles.bookingSection}>
101         <BookingSection
102           selectedSeats={selectedSeats}
103           handleBooking={handleBook
104         />
105       </div>
106     </div>
107   </div>
108 )
109
110
```

CSS Code (trainInfoPage.module.css):

```

1 .body {
2   background-color: #fff;
3   margin: 0;
4   padding: 0;
5   min-height: 100vh;
6 }
7
8 .leftSection {
9   flex: 1;
10  display: flex;
11  flex-direction: column;
12  gap: 20px;
13 }
14
15 .trainDetails {
16   flex: 1;
17   padding: 20px;
18   background-color: #fff;
19   border-radius: 8px;
20 }
21
22 .trainRouteMapContainer {
23   width: 500px;
24   margin-top: 170px;
25   margin-left: 30px;
26   height: 350px;
27   position: fixed;
28 }
29
30 .seatMapAndBooking {
31   display: flex;
32   flex-direction: column;
33   padding: 20px;
34   background-color: #fff;
35   flex: 2;
36   margin-left: 200px;
37 }
38
39 .trainSeatMap {
40   margin-bottom: 20px;
41 }

```

Fault Avoidance/Refactoring

We specifically structured our project to be easy to understand, reducing the chances of coding mistakes that might be missed in dense code sections as outlined by the author. The

majority of the web pages have a file under the Pages folder, the Style folder, and any components that the page implements are in other files under the Components folder. This allows each file to be concise and readable. This also reduces the complexity of each section of our application. Having components that are used across multiple pages also allows us to reuse trusted code and avoid rewriting. We also used console logging extensively, for both the frontend and backend while testing and writing to ensure the non-visible aspects of the application were working as expected. Above is a specific example of a place where we refactored the code into multiple files for better readability. The Train Info page implements 4 components, which are in separate files and can be reused across other pages. It also gets styling from the trainInfoPage CSS module.

Input Validation

When taking user input, we utilized input validation to prevent invalid data from entering the database. This is to avoid errors that might happen from the app trying to pull incorrectly formatted data from the database or errors from inserting invalid data into the database itself. A specific example of where we implemented input validation is in the handleSubmit function on the Signup page. The email and password are both checked against a regex before the call is made to the signup endpoint, which writes to the database. If the endpoint was called before any checks were done on the inputted data, the application would try to write data that doesn't match the variable type in the database, which would lead to a Type error.

```

25  const handleSubmit = async () => {
26    const { firstName, lastName, phoneNumber, email, password, confirmPassword } = formData;
27
28    // Validation checks
29    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
30    const passwordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&]).{8,}$/;
31
32    if (!emailRegex.test(email)) {
33      return setError('Invalid email format.');
```

```

129 // Signup endpoint
130 app.post('/api/signup', (req, res) => {
131   const { firstName, lastName, phoneNumber, email, password } = req.body;
132
133   if (!firstName || !lastName || !phoneNumber || !email || !password) {
134     return res.status(400).json({ error: 'All fields are required.' });
135   }
136
137   db.get('SELECT * FROM Users WHERE email = ?', [email], (err, row) => {
138     if (err) {
139       console.error('Database Error:', err);
140       return res.status(500).json({ error: 'Database error.' });
141     }
142     if (row) {
143       return res.status(400).json({ error: 'Email already in use.' });
144     }
145
146     db.run(
147       `INSERT INTO Users (email, password, first_name, last_name, phone_number, created_at, updated_at)
148       VALUES (?, ?, ?, ?, ?, datetime('now'), datetime('now'))`,
149       [email, password, firstName, lastName, phoneNumber],
150       (err) => {
151         if (err) {
152           console.error('Database Error:', err);
153           return res.status(500).json({ error: 'Failed to create account.' });
154         }
155         res.status(201).json({ message: 'Account created successfully!' });
156       }
157     );
158   });
159 });

```

Failure Management

While developing, we were conscious of managing failures or faults that might happen while our application was running to not disrupt the user experience. Below, you can see the endpoint for the ticket table which handles errors by returning the status. This does not disturb the user but is still accessible in the web console for the developer to find later while fixing the issue. The user is notified on the page of an error with a popup or message. The application continues running. For example, if the logout failed, the detailed message would be logged to the console and written in the status of the request, and the user would be alerted that an error occurred while logging out.

```

361 // Logout endpoint
362 app.post('/api/logout', (req, res) => {
363   if (req.session) {
364     req.session.destroy((err) => {
365       if (err) {
366         console.error('Error destroying session:', err);
367         return res.status(500).json({ error: 'Failed to log out. Please try again.' });
368       }
369
370       // Clear the session cookie
371       res.clearCookie('connect.sid', { httpOnly: true, secure: false, sameSite: 'Lax' });
372       res.status(200).json({ message: 'Logged out successfully.' });
373     });
374   } else {
375     res.status(400).json({ error: 'No active session to log out.' });
376   }
377 });

```

```

1  import React from 'react';
2  import NavigationButton from '../components/navigationButton.js';
3
4  const LogoutButton = () => {
5    const handleLogout = async () => {
6      try {
7        const response = await fetch('http://localhost:3000/api/logout', {
8          method: 'POST',
9          credentials: 'include', // Include session cookies
10         });
11
12         if (response.ok) {
13           console.log('Logged out successfully.');
```

```

14           window.location.href = 'http://localhost:3001/';
15         } else {
16           const errorData = await response.json();
17           console.error('Logout failed:', errorData.error);
18           alert('Logout failed: ' + errorData.error);
19         }
20       } catch (error) {
21         console.error('Error during logout:', error);
22         alert('An error occurred while logging out.');
```

```

23       }
24     };
25
26     return (
27       <NavigationButton
28         text="Logout"
29         path="/"
30         style={{ backgroundColor: 'red', color: 'white' }}
31         onClick={handleLogout}
32       />
33     );
34   };
35
36   export default LogoutButton;

```

Program Complexity

Our program, being a web application, implements many useful packages to function. Pictured below:

```

|— @react-google-maps/api@2.20.3
|— @stripe/react-stripe-js@2.9.0
|— @stripe/stripe-js@4.10.0
|— axios@1.7.7
|— body-parser@1.20.3
|— bootstrap@5.3.3
|— cookie-parser@1.4.7
|— cors@2.8.5
|— cross-env@7.0.3
|— date-fns@4.1.0
|— dotenv@16.4.5
|— express-session@1.18.1
|— express@4.21.1

```

```
— mysql@2.18.1
— qrcode@1.5.4
— react-bootstrap@2.10.5
— react-dom@18.3.1
— react-native-svg@15.9.0
— react-qr-code@2.0.15
— react-qr-code-logo@3.0.0
— react-router-dom@6.28.0
— react-scripts@5.0.1
— react-select@5.8.3
— react@18.3.1
— sqlite3@5.1.7
— stripe@17.3.1
```

Implementing packages allows us to do many things that would be far beyond the scope of this application and our technical abilities to develop, everything from dynamically displaying maps between stations (google-maps-api) to allowing us to host frontend and backend applications concurrently and sending requests between them securely (cors). However, this does significantly increase the complexity of the program and requires our application to depend on code that is not developed and therefore not fixable by our team. This is a tradeoff we considered between functionality and reliability.

We do not have any particularly complex decisions in the program, partially by design and partially from the nature of our project not necessarily requiring complex computational decisions. We do not have any deeply inherited relationships between our structures, again because of the nature of this project. We also avoided complexity in our data and database by restricting the size of the variables, and only storing data that is necessary for the application. Because we use data from Amtrack, there was no need to store data about the actual trains in our database at all. This allows us to only have two tables for the database, Users and Tickets, when we originally expected to also need a local database that pulled and stored train data. There are approximately 200 trains running at any given time with several data points that would need to be stored for each, so this design change saved us significant storage space and complexity.

```
15 CREATE TABLE IF NOT EXISTS Users (  
16     user_id INTEGER PRIMARY KEY AUTOINCREMENT,  
17     email VARCHAR(100) UNIQUE NOT NULL,  
18     password VARCHAR(100) NOT NULL,  
19     first_name VARCHAR(100) NOT NULL,  
20     last_name VARCHAR(100) NOT NULL,  
21     phone_number VARCHAR(20),  
22     card_number INTEGER,  
23     billing_address TEXT,  
24     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
25     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
26 )  
27 `);  
28  
29 db.run(`  
30 CREATE TABLE IF NOT EXISTS Tickets (  
31     ticket_id INTEGER PRIMARY KEY AUTOINCREMENT,  
32     user_id INTEGER NOT NULL,  
33     train_id INTEGER NOT NULL,  
34     origin VARCHAR(100) NOT NULL,  
35     destination VARCHAR(100) NOT NULL,  
36     departure_time DATETIME NOT NULL,  
37     arrival_time DATETIME NOT NULL,  
38     seat_number VARCHAR(60),  
39     qr_code TEXT,  
40     price DECIMAL(4, 2),  
41     purchase_time DATETIME DEFAULT CURRENT_TIMESTAMP,  
42     scanned BOOLEAN DEFAULT 0,  
43     FOREIGN KEY (user_id) REFERENCES Users(user_id)  
44 )  
45 `);
```

Chapter Six: Security

PRIMARY AUTHOR: **Andrew Martens**

CO-SIGNERS: Rachel Roach
Evan Myran
Cadence Stewart
Tyler Safranski

Introduction

Throughout the development of Train Track, we implemented a variety of security features to help protect users. As developers, we understood the application handles sensitive data such as payment and ticket information. If a malicious user were to discover that information they could gain access to another person's payment method or find out where someone might be based on ticket information. To uphold security integrity many features were implemented to protect users against various types of cyber attacks making our application safe to use. This chapter walks through a typical user interaction while describing the steps we took to protect them throughout their experience with our application.

Sign-in

The “root” page of our application is our login page. Assuming this is where a new user starts interacting with our application, they would have the option to navigate to the sign-in page to create a Train Track account. We developed our sign-in page carefully to validate user input for fields deemed more important than others. For example, the password field has a complexity requirement only allowing passwords containing at least 8 characters and at least one uppercase letter, one lowercase letter, one number, and one special character. Another important field is the email input field. Since our database requires unique email addresses, we often use the user's email in SQL lookups when searching for other user-related information. Since the email field is used as a parameter in SQL lookups, it is vulnerable to SQL injection attacks so we decided to implement input validation to ensure all emails going into our database are safe. Upon clicking the sign-in button, assuming the user enters valid information, an account is created within our database. If time had allowed, we would have implemented the *bcrypt* package to hash passwords before they enter the database to store them more securely.

Login & Secure Sessions

After signing in, the user is redirected to the login page. We chose to navigate the user to the login page since clicking the login button creates a secure session for the user. We used the *express-session* package to create a secure session for each user allowing us to authenticate user sessions by setting and storing certain information on our backend server. This protects user data by authenticating each session before allowing access to sensitive endpoints like profiles and purchase tickets. Another feature we would like to implement in the future is two-factor

authentication by using the *node-2fa* package. Unfortunately, we didn't have quite enough time to implement this feature though.

Payment Security

Let's assume a user wants to purchase a ticket. There are many security measures involved with electronic payments which is why instead of creating our algorithms for security we handle user payments with Stripe's trusted API to ensure all possible security measures are taken. When using Stripe's API we made sure to keep our `STRIPE_SECRET_KEY` hidden using the *dotenv* package to protect against unauthorized accesses. Other than that we could essentially let Stripe handle additional security measures during transactions. Stripe is fully Payment Card Industry Data Security Standard (PCI-DSS) compliant at the highest level meaning they follow strict standards when processing, storing, and transferring credit card information. Stripe also uses tokenization meaning sensitive card data is never stored on our servers. Instead, card data is securely sent to Stripe, which returns a token that can be used for processing payments without ever exposing the actual card details. Stripe's other security features include end-to-end encryption and a Radar system that uses machine learning models to detect and prevent fraud. For a full dive into Stripe's security features the link to their detailed security page is here.

Other Security Features

There are some other minor security features we included dealing with various issues. One issue we ran into was the files we should upload to GitHub, our source control application. We implemented different APIs and had sensitive data that we knew shouldn't be uploaded to our repository on GitHub. To ensure our sensitive data wasn't showing up on our repository, we used a `.gitignore` file to protect certain files from being uploaded to the internet. Some sensitive information we kept local includes our Train Track database, the `.env` file, and files containing our local properties. Another minor feature we added about a user's session, is a timeout that automatically logs a user out after one hour. This protects the user from themselves if they forget to log out or if they leave the browser open when they are away from their device. Lastly, it's important to mention that since our application is web-based we would ideally obtain an SSL certificate allowing us to use HTTPS instead of HTTP. HTTPS is essential for web applications to be trusted and secure. If our goal was to deploy a fully functional application we would 100% purchase an SSL certificate for an extra layer of encryption, but due to the nature of this project, we didn't go that route.

Influences From Chapter 7

Chapter 7 in our textbook, "Engineering Software Products" by Ian Sommerville discusses many security issues and concepts that developers should be aware of when developing a software application. This chapter gave our team valuable insight into vulnerabilities we needed to be conscious of when designing our software. Topics such as encryption, authentication, and authorization were all security measures that our group wanted to incorporate

into development. Unfortunately, we ran out of time and could only implement authorization by creating secure server sessions for each user. Sommerville also discussed specific cyber attacks developers need to prevent. Among all the attacks he mentions, SQL injection attacks are one attack we specifically protected against. As mentioned in the sign-in section, we used strict input validation to make sure our SQL statements are safe when using input for SQL queries. The author gives an example where a developer uses the following SQL command:

```
“SELECT * FROM AccountHolders WHERE accountnumber = “ + accNum + “;”
```

The code above selects each row in the AccountHolders table that matches a field, accNum, that is inputted by the user. The author then explains what would happen if the user inputs the account number as “10010010’ OR ‘1’ = ‘1’”. If this were a valid input the resulting SQL code would become,

```
SELECT * from AccountHolders WHERE accountnumber = ‘10010010’ OR ‘1’ = ‘1’;
```

Since the final condition is always true, each row in AccountHolders would be returned and displayed to the malicious user. In our application, we have a similar call to our database that returns rows based on the user’s email address which is an input in the sign-in page. While developing this page, we realized this was an area that could be exploited. To ensure safety within our SQL commands that use the user’s email we implemented strict input validation. The input validation we implemented doesn’t allow spaces or other special SQL characters that could affect our command.

Conclusion

During the development of this project, we identified areas or potential problems and took appropriate steps to keep users safe when interacting with our application. The use of npm packages and APIs in our backend helped us integrate trusted safety practices into our project without the need to invent our algorithms, saving us time and convenience. On the front end, we used input validation when necessary and made sure each user session was valid and could time out after a set amount of time using cookies. This experience has significantly deepened our understanding of user data security, and with more time, we would have incorporated additional safety measures.

Chapter Seven: Software Testing

PRIMARY AUTHOR: **Tyler Safranski**

CO-SIGNERS: Cadence Stewart
Rachel Roach
Andrew Martens
Evan Myran

Our Testing Approach

Our approach to testing our program relied heavily on teamwork between team members. To avoid an abundance of errors in our program, each member created a specific branch to work and test on. On these branches, we worked on small sections of code at once, made sure those sections worked well with the rest of the code already implemented and then committed those portions once they were complete. When the entirety of that portion of the code was completed, we pushed our branches to the main branch to be incorporated officially into the rest of our program. If a problem persisted and the original person working on that portion of the code couldn't figure out the problem, then we would ask other team members to assist in fixing the problem. Typically, these problems occurred in the branches, the main, on our devices, and with importing tools like libraries. With specific systems, we needed to make sure that all team members had the correct dependencies and ports for the system to work, so before someone worked with these programs, they would send the dependency, how to install it, and the documentation involved with that program if someone else was intending on utilizing that resource as well. This was done to prevent any errors caused by someone not having something necessary to the program installed occurring before working on and testing the program.

Testing Process

When new code was implemented into the system, the original member working on the code collaborated with other team members to ensure that that code would merge seamlessly with the code already committed to our main branch in GitHub. This was to prevent the loss of large portions of code during the merging process. If someone was not careful, then they could easily override vital portions of code necessary for the program to run. Additionally, we collaborated frequently and reassessed the errors ran into to keep group members from working on unnecessary or outdated portions of code. After the code was pushed to the main branch, one person would compile the program and ensure everything ran smoothly without any errors appearing. In moments when individual members were having issues compiling the program due to various issues related to their personal devices and port management, code was sent to members individually and implemented from their devices. Providing other members with the code directly was used as a last-resort testing method, but in instances where it was utilized, it quickly got around the issue on the individual programmer's end.

We stressed solid, consistent communication throughout the creation of our program. Communicating with group members allowed us to effectively confer and determine where, what, how, and why specific errors were occurring within our program. This also allowed us to find simpler ways to implement sections of code. Simpler coding portions allow for fewer errors to appear since a lack of complexity means fewer chances for the code to not compile correctly with the rest of the program. It also helped the coders understand how functions operated, which in turn made other people's code easier to work with, especially if someone wasn't originally familiar with that portion of the project.

Finding Small Errors

Before our team tested the program, we ran it through our IDE to catch any potential syntax and formatting errors present in the raw code. This was important because small, simple errors such as typos can branch off into larger issues in the code and prevent the program from compiling. These small errors also needed to be found and resolved before pushing the individual branch being worked on to the main branch where it could break the entire program.

Error Handling

Throughout our website, there are many instances where the user must enter information into a text field that then feeds that information into our linked database. To prevent information that did not meet the parameters for that specific field from getting sent to the database, our team inputted error handling to ensure all information sent to the database was correctly formatted. For example on the signup page where the users must input all the information necessary for creating an account with Train Track, if the user enters an email address that doesn't correspond with what our database receives, an error message appears stating "Invalid email format." When creating or changing a password for an account, the user must input a password that follows the RegEx formatting: one capitalized and lowercase letter, one number, and a special character. If these requirements aren't met, then an error message appears stating that the password created doesn't meet the qualifications necessary for the system. Essentially, if the user has inputted something into an input field that doesn't meet the requirements, an error message appears on the screen telling the user what threw the error. This testing process prevented the user from causing a slew of issues with both the frontend and the backend of our system and ensured consistency throughout the information being read from and into the database.

Avoiding Dependency & Package Issues

We frequently ran into the same errors throughout the project with slight variations between them. Often, these errors are related to the npm functionality in Node.js and other related dependencies involved in the project. Whenever this occurred, we either reinstalled the package that was throwing the error, or we did an npm install to find anything missing that was involved with npm itself. By performing an npm install, the system grabs any missing dependencies from recently merged packages that weren't automatically included when the team

member installed them. Doing this allowed us to inadvertently find anything missing with the installed packages, saving us from future headaches related to incorrectly installing packages.

Code Reviews

During our code reviews, we made sure the individual who worked on the portion being reviewed had a branch of working code that had yet to be implemented into the main program. From there, another team member would go into the branch and review what changes had been made since main was last pulled to the branch. Once everything in the branch worked properly and it had been reviewed by another team member, a merge request was then sent to GitHub where team members would review the code once more and make sure that no errors would occur upon merging the branch with the main system. After the merge cleared, the branch and its changes would then be pushed to the main branch where someone would compile everything the team has done so far and test the new addition. If everything worked out, then we would either go back to working on a branch that was previously created or we would create a new branch that works with a different aspect of the system. Despite the process being slightly inefficient, this process worked best for our team because we had multiple people checking the branches and the main which ensured no errors were pushed to the main or left there.

Additional Info: Design Iterations

CO-SIGNERS: Evan Myran

Cadence Stewart

Rachel Roach

Andrew Martens

Tyler Safranski

These are in-progress diagrams/notes/info that were used during the development of our application.

- I. First Meeting Notes
- II. Site Map (October 25th)
- III. Wireframes
- IV. Database Tables

I. First Meeting Notes

Train Ticketing System

This system is intended to be used by travelers and railroad employees to enable the electronic purchasing of tickets along with the various functions needed by passengers to manage their ticket purchases and board a train for departure.

1. Minimums of a profile (Profile object)
 - a. Username/Email
 - i. Regex - contains the proper requirements for email address &/or username
 - ii. Regex email: "[a-zA-Z0-9_-]+\w+@[a-zA-Z0-9_-]+\w\.[a-zA-Z]{2,}"
 - iii. Register – confirm password
 - b. Unique User ID
 - c. Saved Payments
 - d. Password
 - i. Certain requirements to ensure a certain level of security
 - ii. Recovery system (email confirmation/"forgot password" function)
 - iii. Consider 2FA
 - e. Add a profile page that holds settings for the profile itself
 - i. Logout button
 - ii. Current Tickets
 - f. Database for user information
 - g. Catch for incorrect password/username
2. Ticket Purchasing
 - a. Select date, time and starting/ending location to search for available tickets

- b. Select the train you want to purchase a ticket for (train object?)
 - c. Show relevant train information information:
 - i. Num of tickets available
 - ii. Seats available on train
 - d. Enter card detail
 - i. Store card information on file if user requests
 - e. Verify whether the card entered is valid
 - f. Potentially have a midnight train that's always full to demonstrate how if the train is full you cannot buy a ticket for it
 - g. Make it so one less seat is available on the train after purchase
 - h. Potentially add billing address
 - i. Link venmo and paypal to potentially pay
- 3. Ticket/Train Information
 - a. Database for train information
 - b. QR code generations
 - i. Once you buy the ticket, you can click on the purchase to open the QR code to scan
 - c. Refundability/ticket cancellation
 - i. Go into a ticket, there's a button to confirm to cancel, "are you sure?", send email to say that ticket has been canceled
 - ii. Database for past tickets, purchase info, QR code, etc
 - d. Send receipts to email once ticket is purchased
 - i. Store receipts on a purchase history page on user profile?
- 4. GUI:
 - a. Modern design, easy to understand, simple
 - b. Home page, Ticket page, Profile page, Sign up, Log in

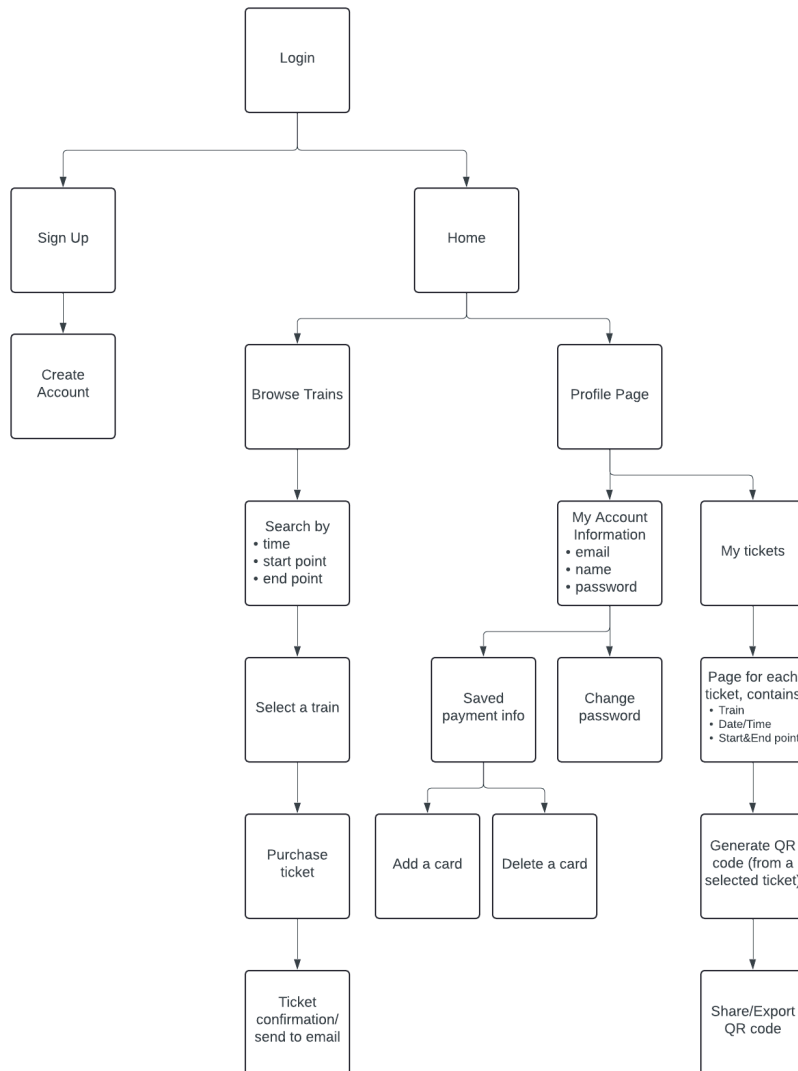
II. Site Map (October 25th)

Necessary Pages for the Website:

- When you initially open website:
 - **Log in/Sign up page**
 - Form to create an account
 - Form to login
- After you've logged in
 - **Home Page**
 - **Browse Train Tickets**
 - Search by starting point/destination?
 - Sort by time?
 - **Profile Page**
 - Contains user info (Username, Payment info,
 - Change password
 - Payment Information (if you want to add a card on file)

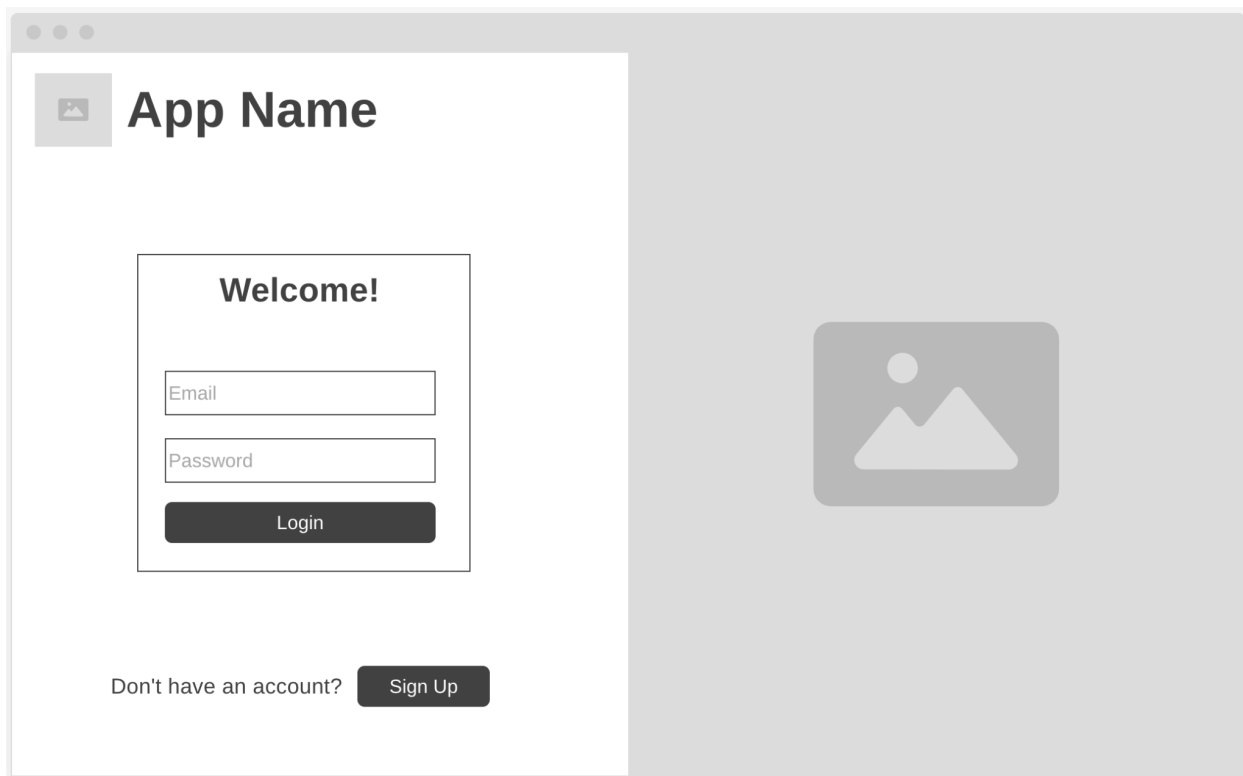
- **Current Tickets Page**
 - Shows purchased tickets (able to be used)
 - QR Code
 - Share/Export button?
 - Has info like Date/Time, Start/Destination, Train identifier
- Purchasing:
 - Select Ticket
 - (click on a train in home page and it will bring you to this)
 - Purchase Ticket form

Site Map Diagram:



III. Wireframes

Login



A wireframe for a login page. The page is divided into two main sections. The left section is white and contains the login form, while the right section is gray and contains a large placeholder for a profile picture. The login form is titled 'App Name' and includes a 'Welcome!' message. It features two input fields for 'Email' and 'Password', followed by a 'Login' button. Below the form, there is a link 'Don't have an account?' and a 'Sign Up' button. The gray section on the right has a large square placeholder with a mountain and sun icon.

App Name

Welcome!

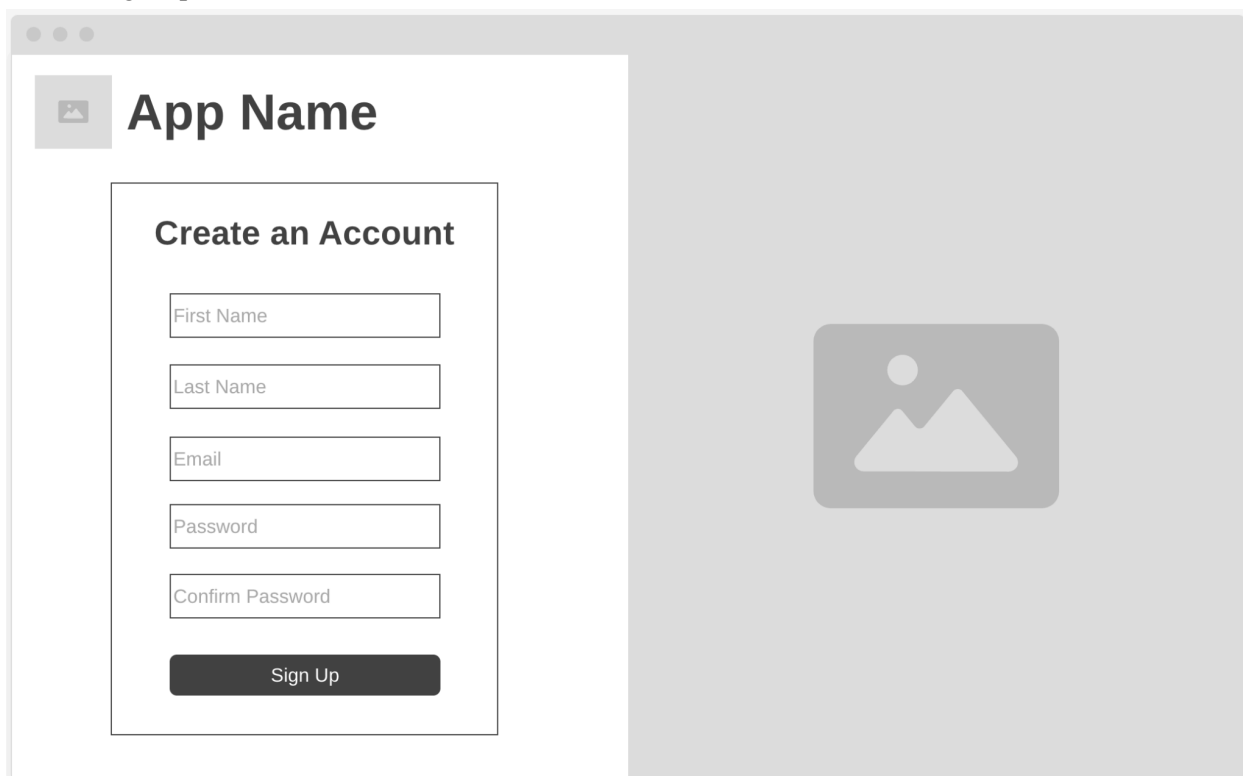
Email

Password

Login

Don't have an account? [Sign Up](#)

Sign Up



A wireframe for a sign-up page. The page is divided into two main sections. The left section is white and contains the sign-up form, while the right section is gray and contains a large placeholder for a profile picture. The sign-up form is titled 'App Name' and includes a 'Create an Account' message. It features five input fields for 'First Name', 'Last Name', 'Email', 'Password', and 'Confirm Password', followed by a 'Sign Up' button. The gray section on the right has a large square placeholder with a mountain and sun icon.

App Name

Create an Account

First Name

Last Name

Email

Password

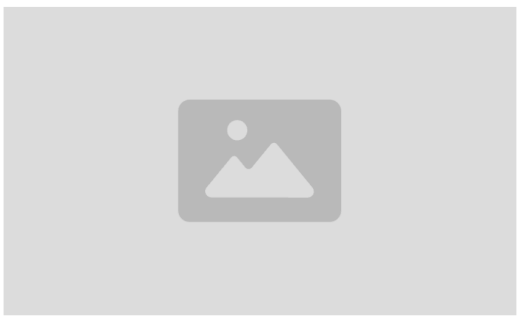
Confirm Password

Sign Up

Home

 App Name

 My Profile



View My Tickets

From

To

1 Traveller

Find Trains

Browse Trains

 App Name

 My Profile

Browse Trains

Search 

Train ID	Origin	Destination	Departure Time	Available Seats	Price
ABC	Sioux Falls	Rapid City	10:00A	24	\$9.99

Train Info

 App Name

 My Profile

ABC

Origin: Sioux Falls

Destination: Rapid City

Departure Time: 10:00A

Open seats: 24

map?

display open seats on train

Book Tickets

Profile - User Info

 App Name

 My Profile

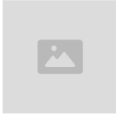
My Profile

User Info

Payment Info

Security

Ticket History



First Name Last Name
User 0001234

Personal Information

First Name:
FN

Email:
xxx@outlook.com

Last Name:
LN

Phone Number:
000-000-0000

Profile - Security

 App Name

 My Profile

My Profile

User Info

Payment Info

Security

Ticket History



First Name Last Name
User 0001234

Security

Email:
xxx@outlook.com

Change Password

Profile - Payment info

 App Name

 My Profile

My Profile

User Info

Payment Info

Security

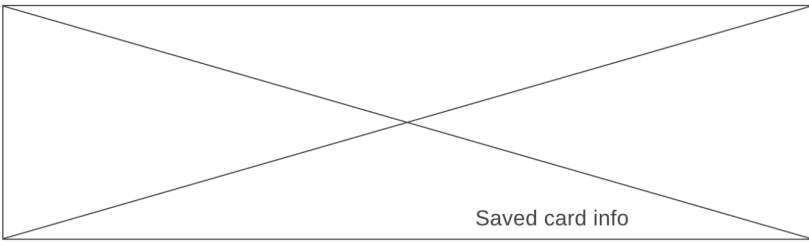
Ticket History



First Name Last Name
User 0001234

Saved Cards

Add a CardDelete a Card


Saved card info

My tickets

 App Name

 My Profile

Purchased Tickets

Train ID	Origin	Destination	Departure Time	Departure Date	View Ticket
ABC	Sioux Falls	Rapid City	10:00A	1/1/2024	View

Ticket example

 App Name

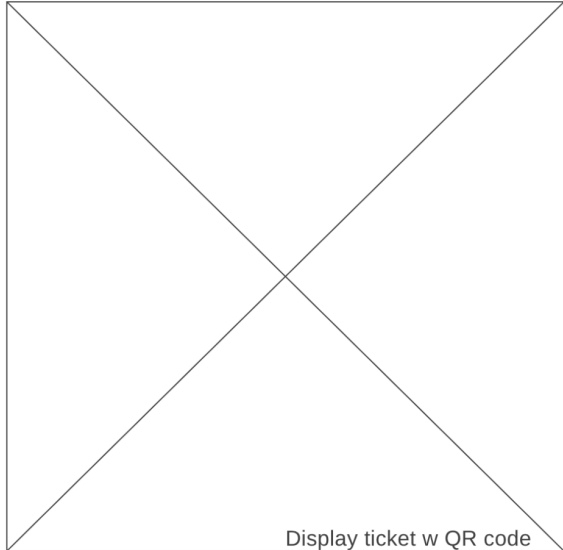
 My Profile

Ticket #0001

Origin: Sioux Falls
Destination: Rapid City
Departure Date: 1/1/2024
Departure Time: 10:00A



map?



Display ticket w QR code

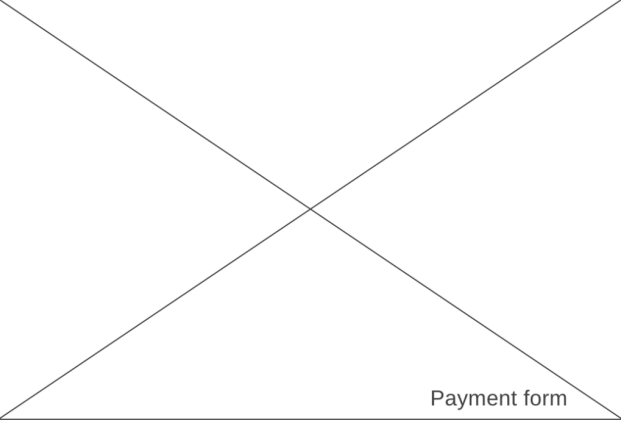
Purchase ticket

 App Name

 My Profile

Check Out

Select Payment Option



Payment form

Your Cart



Ticket #0001
Info (Seat Number, Origin,
Destination, Time, Date)
\$9.99

Order Summary

Subtotal:	\$9.99
Handling fee:	\$3.99
Tax:	\$2.99
Total:	\$16.97

IV. Database Tables

Database Password: software123!

Required

Unique

Required & Unique

Foreign key

User Table

Column Name	Data Type	Description
userid	Int (primary key)	Unique identifier for each user
email	varchar(100) UNIQUE	Unique email (created during account creation) (Needs input validation)
password	varchar(100)	Password (hashed for security?) (Needs input validation)
first_name	varchar(100)	First name (Needs input validation)
last_name	varchar(100)	Last name (Needs input validation)
phone_number	varchar(20)	Phone number (Needs input validation)
card_number	int	16-digit card number.
billing_address	text	Billing address tied to account for payment (Needs input validation)
created_at	timestamp	When account was created
updated_at	timestamp	When account last had changes/updates

Tickets Table

Column Name	Data Type	Description
ticketid	Int (primary key)	Unique identifier for each ticket
user_id	Int (fk)	Links ticket to user that purchased it
train_id	Int (fk)	Links ticket to the train
departure_time	Datetime (fk)	Departure time
arrival_time	Datetime (fk)	Arrival time
seat_number	varchar(60)	Stores the seat number corresponding to the ticket

qr_code	?	Stores some sort of reference to the corresponding QR
price	decimal(4,2)	Will be an example price like \$9.99
purchase_time	datetime	Time/date ticket was purchased
scanned	boolean	True for if it's been used, and false for when it's able to be scanned and show the qr code.

TrainData Table

Column Name	Data Type	Description
trainid	Int (primary key)	Unique identifier for each train
origin_station	varchar(100)	Origin station
destination_station	varchar(100)	Destination station
departure_time	datetime	Departure time
arrival_time	datetime	Arrival time
available_seats	int	Generates randomly to simulate seats being purchased
last_updated	timestamp	Stores the last time data was pulled from amtrack